

AutoLyrics implementation blueprint for Whisper+LoRA singing transcription

Build `whisper-small` with LoRA applied to both encoder and decoder attention, fine-tuned on speaker-disjoint chunks of NUS-48E, and target a >15% relative WER reduction on held-out singers. Singing is primarily an *acoustic* out-of-distribution shift — not a linguistic one — so the project's initial assumption of "LoRA on the decoder only" should be revised: the decoder already knows English, it is the encoder that must learn sustained vowels, melodic pitch contours, and music-adjacent acoustic patterns. The NUS-48E corpus is tiny (169 minutes, 12 singers, 20 songs, 48 sung recordings) (`nus`) (`Academia.edu`) and ships with **phone-level Audacity-style label files rather than TextGrid transcripts**, so word-level text must be reconstructed from the 20 canonical song lyrics. All guidance below is specific, pinned, and written to be executed file-by-file by an AI coding agent.

The NUS-48E dataset and how to access it

NUS-48E (Duan et al., APSIPA 2013) (`dblp`) is distributed informally by the NUS Sound and Music Computing Lab. (`Nus`) There is no official HuggingFace mirror. The de-facto download is a **community-maintained Google Drive folder linked from the Amphion toolkit** at folder id `12pP9uU10HTVANU3IPLnumTJiRjPtVUMx`; fallbacks are OpenDataLab (`opendatalab.com/OpenDataLab/NUS-48E`) and emailing A/Prof. Ye Wang (`wangye@comp.nus.edu.sg`). Usage is **academic/non-commercial** — cite the 2013 APSIPA paper.

The corpus contains **12 singers** (`Academia.edu`) (**4-letter codes: ADIZ, JLEE, JTAN, KENN, MCUR, MPOL, MPUR, NJAT, PMAR, SAMF, VKOW, ZHIY; 6 female / 6 male**), **20 English songs**, and **48 annotated song-singer pairs** (each singer sang 4 of the 20 songs, producing both a *sung* and a *read/spoken* version of identical lyrics). (`nus`) Total audio: **115 min sung + 54 min spoken** $\approx$ **1-2 GB on disk**. All WAVs are **44.1 kHz, 16-bit, mono**, captured with an Audio-Technica 4050 (`nus`) in a sound-proofed studio.

The directory layout is `{SingerCode}/{sing|read}/{SongID}.{wav,txt}` (`GitHub`) where `{SongID}` is a numeric string `01-20`. The `.txt` files are **Audacity label-track text**, not Praat TextGrid — one line per phoneme with format `<start_s>\t<end_s>\t<phone>`, using the **CMU 39-phone set plus `s11` and `sp` markers**. Sung annotations are manually labeled; spoken annotations were force-aligned with a WSJ0-trained GMM-HMM. (`nus`) There is **no word-level or line-level timing** in the shipped release.

This has a direct consequence for training: Whisper needs orthographic word transcripts, so you must build a `lyrics_map: dict[song_id, str]` by transcribing the 20 canonical lyrics once (the paper's song list and lyrics are posted at singingevaluation.wordpress.com/2012/11/22/songs-to-pick/) (`nus`) and then segment both audio and lyric text together using silence (`(sil)`/`(sp)`) markers in the phone tier.

Splitting strategy. With 48 recordings, random splits leak the same song lyrics into train and test. Use a **speaker-disjoint split**: train on 8 singers, validate on 2, test on 2, balanced by gender (e.g., test = `(JLEE + KENN)`, val = `(SAMF + VKOW)`). For publication-grade results also report **12-fold leave-one-singer-out cross-validation**. Additionally, report on *sung* and *read/spoken* splits separately — NUS-48E's parallel design makes this a first-class analysis.

Base model selection and the LoRA target-module decision

The recommended base is `openai/whisper-small` (244M parameters, 12 encoder + 12 decoder layers, 768-dim, 80 mel bins). (`huggingface`) Whisper overfits quickly on <10h datasets; the HuggingFace-canonical Sanchit Gandhi fine-tuning reference uses whisper-small, and community reports (ML6 Dutch study, OpenAI discussions #678 and #85) consistently show medium/large-v2 hallucinating on tiny data. (`Medium`) Keep `whisper-medium` (769M, (`Medium`) `openai/whisper-medium`) (`huggingface`) as a secondary candidate (`Hugging Face`) if baseline zero-shot WER is unacceptable. **Avoid** `whisper-large-v3` (uses 128 mel bins — a subtle gotcha) and `whisper-large-v3-turbo` (its 4-layer decoder undercuts the benefit of decoder LoRA).

Module naming (verified directly against

`transformers/src/transformers/models/whisper/modeling_whisper.py`): every `WhisperAttention` instance exposes `(q_proj, k_proj, v_proj, out_proj)`, (`GitHub`) and FFN blocks are `(fc1, fc2)`. PEFT matches these by suffix across the whole model, so passing `target_modules=["q_proj", "v_proj"]` by default targets both encoder self-attention, decoder self-attention, and decoder cross-attention. One critical implementation detail: under 8-bit loading, the encoder `(Conv1d)` stem (`(model.model.encoder.conv1)`) needs a forward hook to propagate gradients (`(output.requires_grad_(True))`); omitting it silently breaks training.

The project spec asks for "LoRA applied to the decoder." The literature argues for the opposite. Gao et al. (arXiv:2506.02339, 2025) fine-tune Whisper with LoRA for automatic lyrics transcription targeting the **encoder**, using a consistency loss between separated vocals and mixtures. (`arXiv`) S2-LoRA (arXiv:2309.11756) on acoustically OOD child speech finds encoder modules dominant. The LoRA-Whisper paper (arXiv:2406.06619) applies LoRA to `{W_q, W_k, W_v, W_fc}` in **both** encoder and decoder and achieves 18.5% relative WER reduction. (`arXiv`) Our recommendation is therefore to **apply LoRA to both encoder and decoder attention projections**

and run a small ablation (decoder-only vs encoder-only vs both) to satisfy the project's stated comparison.

LoRA hyperparameters for 48 songs: `(r=8)`, `(lora_alpha=16)`, `(lora_dropout=0.1)`, `(bias="none")`, `(target_modules=["q_proj", "k_proj", "v_proj", "out_proj"])`, `(task_type="SEQ_2_SEQ_LM")`. The conservative `(r=8)` (vs community default `(r=32)`) directly addresses the overfitting risk of a tiny dataset; scale to `(r=16)` only if the training loss plateaus above evaluation WER targets.

Data preprocessing and HuggingFace pipeline

Convert NUS-48E to a `(DatasetDict)` of three speaker-disjoint splits. Whisper hard-caps at **30-second inputs (padded to exactly 3000 mel frames) and 448 decoder tokens**, so songs of up to ~200s must be chunked. Target **20–25 s chunks aligned to `(sil)`/`(sp)` boundaries** in the phone tier so words are never cut. For each chunk compute the 80-bin log-Mel spectrogram via `(WhisperFeatureExtractor)` `(n_fft=400)`, `(hop_length=160)`, `(win_length=400)` and tokenize the corresponding lyric slice with `(WhisperTokenizer(language="English", task="transcribe"))`. `(LearnOpenCV)`

```
python
```

```

# src/autolyrics/data/nus48e.py (core sketch)
import torchaudio, torchaudio.transforms as T, numpy as np, soundfile as sf
from datasets import Dataset, DatasetDict, Audio

TARGET_SR, CHUNK_SEC, MAX_SEC, MIN_SEC = 16000, 22.0, 28.0, 3.0

def load_wav_16k_mono(path):
    wav, sr = torchaudio.load(path)
    if wav.shape[0] > 1: wav = wav.mean(0, keepdim=True)
    if sr != TARGET_SR:
        wav = T.Resample(sr, TARGET_SR)(wav)
    return wav.squeeze(0).numpy().astype("float32")

def parse_nus_label(path):
    segs = []
    for line in open(path):
        p = line.split()
        if len(p) >= 3:
            segs.append((float(p[0]), float(p[1]), p[2]))
    return segs

def chunk_by_silence(audio, segs, lyric_words, sr=TARGET_SR,
                    target=CHUNK_SEC, max_s=MAX_SEC, min_s=MIN_SEC):
    """Walk phones, break at sil/sp once >= target seconds accumulated.
    `lyric_words` is the full song-lyric list; word indices advance as we
    cross sp markers in the phone tier, so each chunk gets a word slice."""
    chunks, cur_start, w_i, chunk_words = [], None, 0, []
    for (a, b, ph) in segs:
        if ph.lower() in ("sil", "sp"):
            if cur_start is not None and (a - cur_start) >= target:
                chunks.append((audio[int(cur_start*sr):int(a*sr)],
                              " ".join(chunk_words), cur_start, a))
                cur_start, chunk_words = None, []
            if ph.lower() == "sp" and w_i < len(lyric_words):
                if chunk_words == [] and cur_start is None:
                    pass # between chunks, still advance
                else:
                    chunk_words.append(lyric_words[w_i])
                    w_i += 1
                continue
            if cur_start is None:
                cur_start = a
            if b - cur_start >= max_s:

```

```

        chunks.append((audio[int(cur_start*sr):int(b*sr)],
                        " ".join(chunk_words), cur_start, b))
        cur_start, chunk_words = None, []
    return chunks

```

After persisting chunks as WAV files on disk, build the HF dataset with

```

Dataset.from_dict({"audio": paths, "transcription": texts, "singer_id":...,
                  "song_id":..., "mode":...}).cast_column("audio", Audio(sampling_rate=16000)).

```

(Medium) Apply the canonical `prepare_dataset` map function that calls

```

feature_extractor(audio["array"], sampling_rate=16000).input_features[0] (Medium)
and tokenizer(text).input_ids, (Medium) (Hugging Face) then .filter(lambda x:
len(x["labels"]) < 448). Set os.environ["TOKENIZERS_PARALLELISM"]="false" before
map(..., num_proc=2) to avoid tokenizer fork hangs.

```

Augmentation policy. SpecAugment is the right default for Whisper (it was used in pretraining):

`T.TimeMasking(40)` $\times 2$ and `T.FrequencyMasking(10)` $\times 2$ applied to the `(80, 3000)` log-Mel tensor inside the collator on training batches only. Mild additive noise (SNR 10-20 dB) and $\pm 10\%$ time-stretch are safe. **Explicitly avoid pitch-shift augmentation:** pitch is content-bearing in singing, and shifting it undermines what the model should learn. No external source separation is applied per project requirements; the model must learn to be robust to whatever accompaniment is present (NUS-48E is a-cappella, so this is effectively free).

The data collator is the canonical `DataCollatorSpeechSeq2SeqWithPadding` from Sanchit Gandhi's blog, modified to mask `-100` for label pads and strip the BOS token if prepended:

```
python
```

```

@dataclass
class DataCollatorSpeechSeq2SeqWithPadding:
    processor: Any
    decoder_start_token_id: int
    def __call__(self, features):
        input_features = [{"input_features": f["input_features"]} for f in features]
        batch = self.processor.feature_extractor.pad(input_features, return_tensors="pt")
        label_features = [{"input_ids": f["labels"]} for f in features]
        labels_batch = self.processor.tokenizer.pad(label_features, return_tensors="pt")
        labels = labels_batch["input_ids"].masked_fill(
            labels_batch.attention_mask.ne(1), -100)
        if (labels[:, 0] == self.decoder_start_token_id).all().cpu().item():
            labels = labels[:, 1:]
        batch["labels"] = labels
        return batch

```

Training configuration with PEFT and BitsAndBytes

Load the base model with 8-bit quantization (4-bit NF4 only if VRAM-constrained; 8-bit is more stable for Whisper), prepare for k-bit training, attach the LoRA adapter, and train with

`Seq2SeqTrainer`. Two flags are **mandatory** for PEFT+Trainer compatibility:

`remove_unused_columns=False` and `label_names=["labels"]` — otherwise the Trainer strips the wrong columns because `PeftModel.forward` has a different signature than the base. Set `model.config.use_cache=False` during training (incompatible with gradient checkpointing) and flip it back for inference.

python

```
# src/autolyrics/models/whisper_lora.py
import torch
from transformers import (WhisperForConditionalGeneration, WhisperProcessor,
                          BitsAndBytesConfig, Seq2SeqTrainingArguments)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

MODEL_ID = "openai/whisper-small"
processor = WhisperProcessor.from_pretrained(MODEL_ID, language="English", task="tra

bnb = BitsAndBytesConfig(load_in_8bit=True)
model = WhisperForConditionalGeneration.from_pretrained(
    MODEL_ID, quantization_config=bnb, device_map="auto")
model.config.forced_decoder_ids = None
model.config.suppress_tokens = []
model.config.use_cache = False
model.model.encoder.conv1.register_forward_hook(
    lambda m, i, o: o.requires_grad_(True))          # CRITICAL under 8-bit
model = prepare_model_for_kbit_training(model)

lora = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.1, bias="none",
                  task_type="SEQ_2_SEQ_LM",
                  target_modules=["q_proj", "k_proj", "v_proj", "out_proj"])
model = get_peft_model(model, lora)
model.print_trainable_parameters()    # ~0.4-1% trainable
```

The training arguments for ~500-800 training chunks (typical for NUS-48E after chunking) should use `learning_rate=1e-4` with linear decay, `warmup_steps=50`, `num_train_epochs=8-10` with **early stopping on eval WER** (Whisper overfits on small data after ~5 epochs [GitHub](#) per OpenAI discussion #678), `per_device_train_batch_size=8` with `gradient_accumulation_steps=2` (effective 16), `gradient_checkpointing=True`, `bf16=True` on A100/4090 else `fp16=True` on T4/V100, `predict_with_generate=True`, `generation_max_length=225`, [GitHub](#) and `metric_for_best_model="wer"` with `greater_is_better=False`. [GitHub](#) For small-data runs, set `eval_steps=50` and `save_steps=50` with `save_total_limit=3`.

The resulting adapter is **5-20 MB** (vs the 967 MB base checkpoint), which is the entire point of LoRA.

Evaluation with jiwer and Whisper's English normalizer

Version caveat: `jiwer >= 4.0` (Feb 2025) removed `compute_measures` (GitHub) and renamed `truth→reference`, (GitHub) breaking `evaluate.load("wer")` from HuggingFace. Either pin `jiwer==3.0.5` (simplest, compatible with `evaluate`) or use `jiwer >= 4.0` directly via `jiwer.wer()`, `jiwer.cer()`, and `jiwer.process_words()`. The blueprint pins `jiwer==3.0.5` in requirements.

For paper-comparable numbers, normalize both reference and hypothesis through Whisper's `EnglishTextNormalizer` (available from `transformers.models.whisper.english_normalizer`) before computing WER. It (GitHub) lowercases, expands contractions (`I'm→i am`), (PyPI) spells out numbers, and strips punctuation. Report **two WER figures**: raw orthographic WER and normalized WER. The `compute_metrics` function plugged into `Seq2SeqTrainer` should do the decoding, replace `-100` with `pad_token_id`, run both through the normalizer, drop pairs where the reference becomes empty, and return `{"wer": ..., "cer": ..., "wer_ortho": ..., "cer_ortho": ...}`.

Target. Published Whisper WERs on a-cappella / polyphonic singing benchmarks range from **15% (Jamendo + LyricWhiz ensemble) to 38% (DSing baseline)**. On NUS-48E there is no published Whisper benchmark as of April 2026, so the baseline is part of the project contribution. Realistic expectations: zero-shot `whisper-small` on NUS-48E *sung* will likely land at **40–70% WER**, while *spoken* will be **10–25% WER** — closing that "singing gap" is the story. After LoRA fine-tuning, a reasonable sung WER target is **25–50%** with **>15% relative reduction** (the project goal). Relative reduction = $(\text{WER}_{\text{base}} - \text{WER}_{\text{ft}}) / \text{WER}_{\text{base}} \times 100$.

Beyond overall WER/CER, compute and report **per-singer WER**, **per-song WER**, and **sung-vs-spoken WER** breakdowns — the last is unique to NUS-48E and illuminates whether fine-tuning helps sung output without regressing spoken output. The `src/autolyrics/evaluation/report.py` module should write a JSON + Markdown report under `outputs/reports/` with all four axes and a prediction dump CSV for qualitative inspection.

Gradio demo architecture

For the inference demo, **do not re-quantize to 8-bit** — a documented PEFT discussion (#477) shows 8-bit Whisper inference is ~5× slower and more hallucinogenic than fp16. (GitHub) Load the base in fp16, wrap with `PeftModel.from_pretrained(base, adapter_path)`, call `model.merge_and_unload()` to fuse LoRA weights into the base for speed, then build an ASR `pipeline` with `chunk_length_s=30, stride_length_s=5` for long-form audio. (Hugging Face)

Pass `generate_kwargs={"language":"english","task":"transcribe","num_beams":1}`; the older `forced_decoder_ids` pattern has known bugs through the pipeline.

The UI is `gr.Blocks` (not `gr.Interface`) because the project requires **side-by-side baseline vs fine-tuned** comparison. A single `gr.Audio(sources=["microphone","upload"], type="filepath", :antCitation[]{citations="ceb2a1a6-4422-4c8a-afe3-1cd09d73ae6e,979ac726-9da1-48fd-a687-804e9c83361b"} format="wav"))` input feeds both pipelines; `type="filepath"` is preferred over `type="numpy"` because the HF ASR pipeline handles resampling/normalization internally. Two `gr.Textbox` outputs in side-by-side columns show both transcripts. Include a `gr.Examples(...)` block with 3-5 sample NUS-48E clips in `examples/`. For HuggingFace Spaces deployment, use `app.py` at repo root, provide `packages.txt` with `ffmpeg` and `libsndfile1`, and request a T4-small GPU space (or use ZeroGPU for free on-demand A100 slices).

Project directory structure and dependencies

Use a `src/` layout so `autolyrics` is pip-installable, which keeps scripts and notebooks importing the same code paths. The top-level tree:

```

autolyrics/
├─ app.py                                # Gradio demo (root = HF Spaces convention)
├─ packages.txt                          # ffmpeg, libsndfile1 for Spaces
├─ requirements.txt                      # pinned deps
├─ pyproject.toml                       # src/autolyrics package metadata
├─ README.md, LICENSE, .gitignore, Makefile
├─ configs/
│   ├─ base.yaml                        # Hydra defaults list, paths, seed
│   ├─ data/nus48e.yaml                 # paths, sampling_rate, split policy
│   ├─ model/{whisper_small,whisper_medium}.yaml
│   ├─ lora/default.yaml                # r, alpha, dropout, target_modules
│   ├─ training/default.yaml            # lr, epochs, batch, eval cadence
│   └─ evaluation/default.yaml          # metrics, normalization, generation
├─ data/
│   ├─ raw/nus-smc-corpus_48/           # git-ignored; drop the corpus here
│   ├─ interim/                         # split manifests, cleaned lyrics
│   └─ processed/                      # 22-s WAV chunks + HF arrow shards
├─ src/autolyrics/
│   ├─ config.py                       # OmegaConf load + path resolution
│   ├─ data/{nus48e.py, preprocess.py, collator.py}
│   ├─ models/{whisper_lora.py, load.py}
│   ├─ training/{trainer.py, callbacks.py}
│   ├─ evaluation/{metrics.py, report.py}
│   ├─ inference/pipeline.py
│   └─ utils/{audio.py, logging.py, paths.py}
├─ scripts/
│   ├─ prepare_data.py                 # raw -> data/processed
│   ├─ train.py                       # Hydra entry point
│   ├─ evaluate.py                    # baseline + finetuned; writes report
│   └─ export_adapter.py              # push to HF Hub
├─ notebooks/                          # EDA, baseline WER, results analysis
├─ outputs/{checkpoints,logs,wandb,reports}/ # git-ignored
├─ examples/                           # demo audio clips
└─ tests/                              # pytest: data, model, metrics, inference

```

Configuration uses **Hydra + OmegaConf** so hyperparameters compose cleanly (`python scripts/train.py model=whisper_medium lora.r=16 training.learning_rate=5e-5`). An alternative simpler path is a single YAML + argparse, but Hydra pays off the moment you need an ablation sweep (decoder-only vs encoder-only vs both).

Dependencies (`requirements.txt`), compatible Python 3.10/3.11 + CUDA 12.1):

```
torch==2.4.1
torchaudio==2.4.1
transformers==4.46.3
accelerate==1.1.1
peft==0.13.2
datasets==3.1.0
bitsandbytes==0.44.1
librosa==0.10.2.post1
soundfile==0.12.1
jiwer==3.0.5
hydra-core==1.3.2
omegaconf==2.3.0
tensorboard==2.18.0
wandb==0.18.7
gradio==5.5.0
numpy==1.26.4
pandas==2.2.3
tqdm==4.67.0
```

Version compatibility notes verified during research: `(peft>=0.10)` is required for clean Whisper LoRA merging, `(transformers>=4.40)` for reliable `(generate_kwargs={"language":...})` in the ASR pipeline, and `(bitsandbytes)` is for **training only** (omit from the Spaces `requirements.txt`).

Concrete file-by-file build order for the coding agent

The agent should build and test in this sequence to surface issues early:

1. `(src/autolyrics/data/nus48e.py)` — implement `(parse_nus_label)`, `(iter_nus)`, `(chunk_by_silence)`, plus a `(build_lyrics_map())` that loads the 20 canonical song lyrics from `(data/interim/lyrics/<song_id>.txt)`. **Unit-test parsing against one real label file before moving on.**
2. `(scripts/prepare_data.py)` — Hydra entry that loads raw NUS-48E, chunks to ≤ 25 s, persists WAVs to `(data/processed/{train,val,test}/chunk_*.wav)`, writes manifests, and emits the final `(DatasetDict)` (saved via `(ds.save_to_disk)`).
3. `(src/autolyrics/data/collator.py)` — the `(DataCollatorSpeechSeq2SeqWithPadding)` above, plus an optional `(apply_specaug=True)` variant for training batches.
4. `(src/autolyrics/models/whisper_lora.py)` — `(build_model(cfg))` returning the PEFT-wrapped Whisper with the Conv1d hook and 8-bit loading.

5. `src/autolyrics/evaluation/metrics.py` — `compute_metrics(pred)` + `evaluate_detailed(model, ds_split)` that returns the per-singer/per-song/sung-vs-spoken DataFrame.
6. `scripts/train.py` — wires `config` → `dataset` → `model` → `collator` → `Seq2SeqTrainer` with `remove_unused_columns=False`, `label_names=["labels"]`, launches training, saves the adapter to `outputs/checkpoints/`.
7. `scripts/evaluate.py` — loads baseline `whisper-small` and the LoRA-merged fine-tuned model, runs both on the test split, writes `outputs/reports/eval_results.json` and a Markdown summary with the relative-reduction table.
8. `src/autolyrics/inference/pipeline.py` + `app.py` — `build_pipe()` with `merge_and_unload()`, Gradio Blocks UI with side-by-side outputs.
9. `tests/` — smoke tests for chunking determinism, label-pad masking, and end-to-end 1-step training.

Key risks and mitigations

The three highest-priority risks derived from the research are acoustic-domain mismatch, text reconstruction, and overfitting on 48 recordings.

Risk	Mitigation
NUS-48E provides only phone-level timing; Whisper trains on orthographic text	Manually transcribe the 20 canonical song lyrics once into <code>data/interim/lyrics/</code> ; align chunks to lyric word-slices via <code>sp</code> -marker counting, or run Montreal Forced Aligner for word-level TextGrids
Overfitting: 169 min of audio on a 244M-parameter model	Small LoRA rank (<code>r=8</code>), dropout 0.1, SpecAugment, early stopping on eval WER, speaker-disjoint splits
8-bit + PEFT gotchas (frozen Conv1d, <code>predict_with_generate</code> flakiness)	Register <code>conv1</code> forward hook; force <code>language="english"</code> via <code>generate_kwargs</code> ; set <code>use_cache=False</code> for training and <code>True</code> for inference; use <code>remove_unused_columns=False</code>
jiwer 4.x vs <code>evaluate</code> incompatibility	Pin <code>jiwer==3.0.5</code> or call <code>jiwer.wer</code> / <code>process_words</code> directly instead of <code>evaluate.load("wer")</code>
Demo hallucination from 8-bit inference (PEFT disc. #477)	Load inference model in fp16, <code>merge_and_unload()</code> LoRA, never run 8-bit at serving time

Conclusion and the single most important architectural decision

The one decision the coding agent must get right — and the one that deviates from the project's original spec — is **where to apply LoRA**. The brief says "decoder only." The 2024–2025 literature (Gao et al. consistency-loss paper, LoRA-Whisper, S2-LoRA) plus the nature of the task says *encoder plus decoder attention*, because singing is an acoustic OOD problem and the decoder already speaks English. Implement the both-branches configuration as the primary run, and include the decoder-only variant as an ablation in `configs/lora/decoder_only.yaml` — comparing the two is a cheap, scientifically valuable analysis that directly supports the "LoRA vs zero-shot" thesis the project is built around.

With the pinned stack (`whisper-small`, `r=8,  $\alpha$ =16, dropout=0.1`, `lr=1e-4`), 8 epochs with early stopping, 8-bit training, SpecAugment, speaker-disjoint splits, normalized WER via Whisper's English normalizer), a >15% relative WER reduction on NUS-48E sung audio is realistic, the LoRA adapter is <20 MB, the full training run fits on a single T4 in a few hours, and the Gradio demo merges cleanly to fp16 for responsive side-by-side inference. Everything above is concrete enough to code.